

METHODS

* Method : used to execute a block of code.

* Syntax:

1. we need to give a name to method (meaning full name).

→ add () {

}

(mandatory)

2. we need to give return type, if we no need to return any value, we have to use void.

→ void : there is nothing to written.

→ we can return 8 primitive datatypes & non-primitive data types.

→ void add () { (method declaration)

int a = 10;

int b = 20;

int c = a + b;

syso (c);

}

* returntype methodName () { → Method Declaration.

}

→ at the end of the method, we have to return something.

* Method Calling (we have to call that method, to activate that) [with that method name]

→ sum ();

(then it will start working).

→ datatype methodName () { }

↓

datatype variable = methodName ();

→ we have to call that in main method only.

* After that we need to create object to that particular class and after that we can call that method using that class object.

Test t = new Test () ;

t.sum () ; → (Non-Static method)

Ex. 1

package com;

public class Test {

psvm {

void sum () {

int a = 10;

int b = 20;

int c = a+b;

syso (c);

}

Test t = new Test ();

t.sum ();

syso (" Done");

}

}

output: 30

Done

Arguments (or) parameters: (we can give arguments to a method).

void sumTwoNumbers (int a; int b) {

int c = a+b;

syso (c);

}

→ argument/parameters.

output: 25

Done

Test t = new Test ();

t.sumTwoNumbers (10; 15);

syso (" Done");

Ex. 2 package com;

public class Test2 {

psvm {

Test2 t = new Test2 ();

int result = t.sumOfTwoNumbers (10, 20);

syso (result/2);

}

~~void~~ sumOfTwoNumbers (int a, int b) {

int c = a+b;

syso (c);

return c;

[x] Local variable.
cannot access]

output: 30
15

*(we cannot write one
method in another method).

* after return variable! we can't write any thing.
 ↳ This value assigned to that method calling value. (like above example, result)

* we cannot return anything from void return type.

* we can only return values to any datatype rather than void type.

Ex:

package programme;

public class PrimeNumbers {

psvm {

Scanner sc = new Scanner(System.in);

int num = sc.nextInt();

PrimeNumbers obj = new PrimeNumbers();
 obj.prime(num);

~~void return~~

int count = 0;
 void prime(int num) { for (int i = 1; i <= num; i++) {

if (num % i == 0) {

count++

}

}

if (count == 2) {

sysout("Prime Number");

}

else {

sysout("Not a Prime Number");

}

(a)

void prime(int num)

class method

Math.sqrt(num)

* To write square root($\sqrt{\quad}$) in java:

(static method)

↳ [due to call with class name i.e math.]


```

* package com;
public class primeNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt(); // syso("Enter a number");
        PrimeNumbers obj = new PrimeNumbers();
        obj.prime(num);
    }

    void prime(int num) {
        int count = 0;
        for (int i = 1; i <= num; i++) {
            if (num % i == 0) {
                count++;
            }
        }
        if (count == 2) {
            syso("Prime Numbers");
        }
        else {
            syso("Not a Prime Number");
        }
    }
}

```

output:

enter a
number :

12

Not a Prime
Number

Number 100

Case 1 : $1 \rightarrow n \rightarrow n$ times (n iterations) (100 iterations)

Case 2 : $2 \rightarrow n \rightarrow (n-1)$ times (99)

Case 3 : $2 - n-1 \rightarrow (n-2)$ times (98 times)

Case 4 : $2 - n/2 \rightarrow (49 \text{ times})$

Case 5 : $2 - \sqrt{n} \rightarrow (9 \text{ times})$

② package com;

public class primeNumbers {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter a number:");

int num = sc.nextInt();

primeNumbers obj = new primeNumbers();

obj.prime2(num);

}

void prime2(int num) {

int count = 0;

for (int i = 2; i <= num; i++) {

if (num % i == 0) {

count++;

}

}

if (count == 1) {

System.out.print("Prime Number");

}

else {

System.out.print("Not a Prime Number");

}

}

}

Output: Enter a number:

12

Not a prime Number.

→ 13

Prime Number.


```

③ package com;
    public class Prime Number {
        psvm {
            Scanner sc = new Scanner(System.in);
            syso("Enter a number:");
            int num = sc.nextInt();
            Prime Number obj = new primeNumber();
            obj.prime3(num);
        }
        void prime3(int num) {
            int count = 0;
            for(int i=2; i<=num; i++){
                if(num%i == 0){
                    count++;
                }
            }
            if(count == 0){
                syso("Prime Number");
            }
            else {
                syso("Not a Prime Number");
            }
        }
    }
}

```

Output: Enter a number:
12
Not a prime Number.

④ package com;
public class PrimeNumbers {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.println("Enter a number");

int num = sc.nextInt();

PrimeNumbers obj = new PrimeNumbers();

obj.prime4(num);

}

void prime4(int num) {

int count = 0;

for (int i = 2; i <= num/2; i++) {

if (num % i == 0) {

count++;

}

}

if (count == 0) {

System.out.println("Prime Number");

}

else {

System.out.println("Not a Prime Number");

}

Output: Enter a Number

12

Not a prime Number.


```

5 package com;
public class PrimeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        syso("Enter a number:");
        int num = sc.nextInt();
        PrimeNumber obj = new PrimeNumber();
        obj.prime5(num);
    }
    void prime5(int num) {
        int count = 0;
        for(int i = 2; i <= (int) Math.sqrt(num); i++) {
            if(num % i == 0) {
                count++;
            }
        }
        if(count == 0) {
            syso("prime Number");
        }
        else {
            syso("Not a prime Number");
        }
    }
}

```

Output:

Enter a number?

12

Not a prime Number.